Application development for Android

Jetpack Compose

https://developer.android.com/jetpack/compose/documentation

Jetpack Compose

- *Jetpack Compose*, or simply *Compose*
  - Creating screens through code rather than XML layout files
  - Screens based on elements designated as *Composables*
  - "*Do more with less code*"
  - Enables a *Declarative UI Model* implementation

Composable

- A *Composable* is a function that generates a UI element (*widget*), possibly composed by other *Composables*
  - It is annotated with `@Composable`
  - The function can receive parameters to allow for UI customization
  - The content of the function is used to declaratively describe the elements (*widgets*) that will constitute the UI element
  - It should not rely on global variables or random values
  - It should not modify global variables or operate outside of its context (except when delegated by external entities)
  - Composable functions can be executed in any order
  - Composable functions can be executed in parallel

Composable

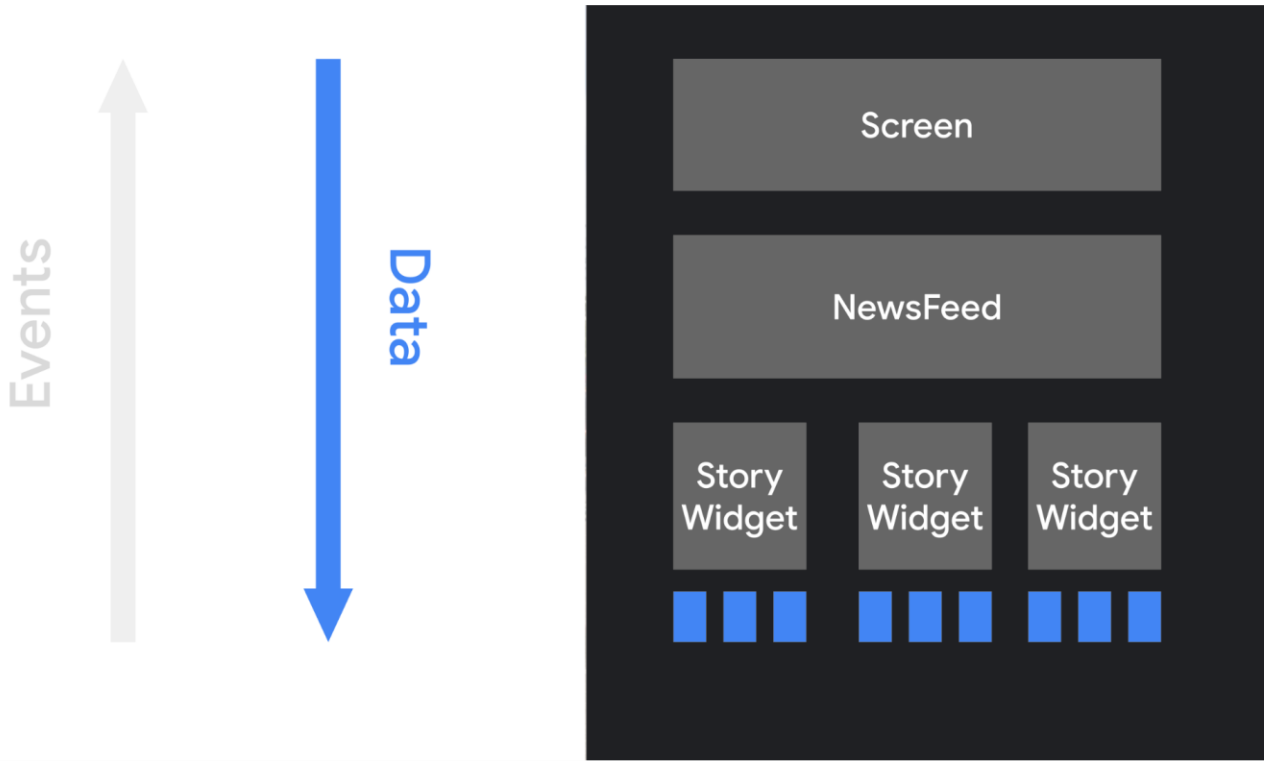
- Example:




```
@Composable
fun Greeting(name: String) {
    Text("Hello $name")
}
```

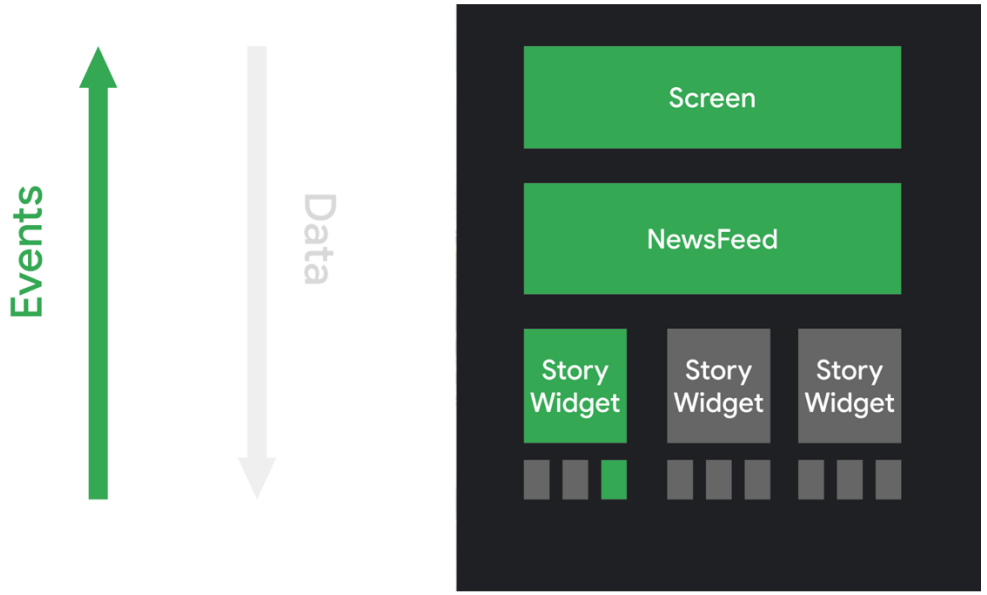
Declarative paradigm

- There are no references to the objects created for later use of getters or setters for their reconfiguration
- The reconfiguration of elements involves calling the Composable function again with different parameters



Declarative Paradigm

- When events are generated, they should be reflected in the application's logic
  - Changing the data can result in a call to the Composable function




```
@Composable
fun ClickCounter(clicks: Int, onClick: () -> Unit) {
    Button(onClick = onClick) {
        Text("I've been clicked $clicks times")
    }
}
```

Dynamic content

- The elements within a Composable function are dynamically created and based on logic
  - Flow control structures can be used to affect the creation or non-creation of elements
  - Flow control structures can be used to iteratively create elements

```
@Composable
fun Greeting(names: List<String>) {
    for (name in names) {
        Text("Hello $name")
    }
}
```

Jetpack Compose phases

- There are three main phases:
  - Composition
    - Compose runs composable functions and creates a description of your UI
  - Layout
    - Layout elements measure and place themselves and any child elements in 2D coordinates, for each node in the layout tree
  - Drawing
    - UI elements are drawn in a Canvas, usually a device screen



Recomposition

- Recomposition is the process of calling a composable function again when inputs change
  - Recomposition of a Composable is typically triggered when a change on a `State<T>` object occurs, in which the Composable is dependent on
  - When Compose recomposes based on new inputs, it only calls the functions or lambdas that might have changed, and skips the rest
  - By skipping all functions or lambdas that don't have changed parameters, Compose can recompose more efficiently

Recomposition

- As certain parts of code may not be executed due to the recomposition functioning, there are procedures that should not be performed within a Composable function, examples:
  - Writing to a property of a shared object
  - Updating an observable in *View Model*
  - Updating shared preferences

State management

- The recomposition of elements is performed based on states
- *Widgets* that rely on states are referred to as *stateful,* as opposed to *stateless widgets*
- States are represented using `MutableState<T>` objects, typically created using the `mutableStateOf` function

```
interface MutableState<T> : State<T> {
    override var value: T
}
```

  - `LiveData` objects can be used to define a state, by using the `LiveData<T>.observeAsState()` method

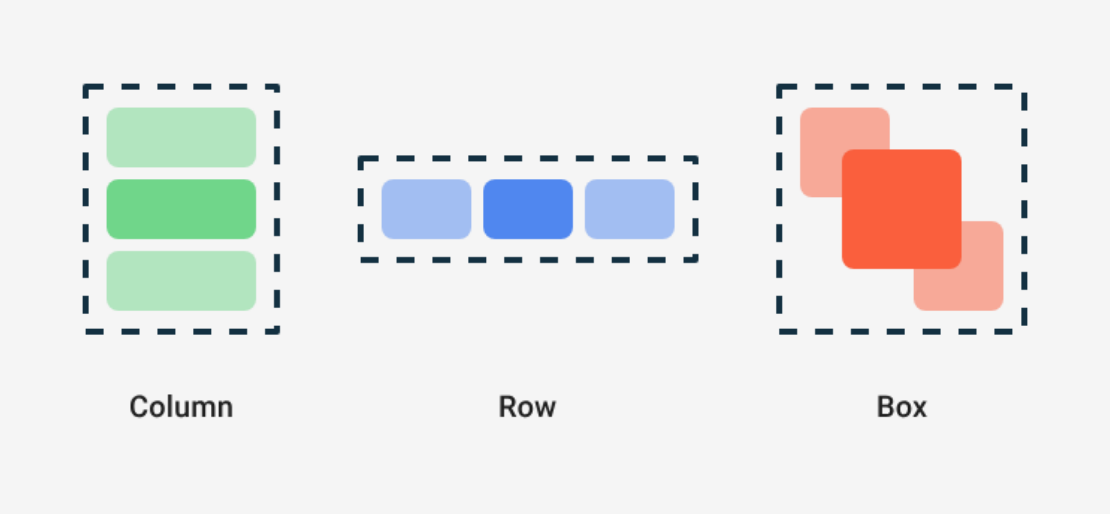
- Ways to create MutableState:
  - `val mutableState = remember { mutableStateOf(default) }`
  - **`var value by remember { mutableStateOf(default) }`**
  - `val (value, setValue) = remember { mutableStateOf(default) }`

Example

```
@Composable
fun BasicExample() {
  var count by remember { mutableStateOf(0) }
  Column (
    verticalArrangement = Arrangement.Center,
    horizontalAlignment = Alignment.CenterHorizontally
  ) {
    Text(text = "Value $count")
    Button(onClick = { count++ }) {
      Text(text = "Increment")
    }
  }
}
```

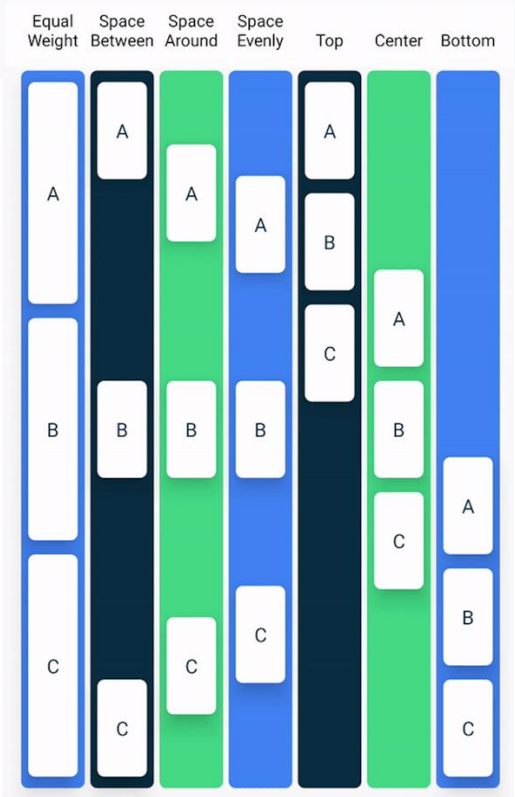
Layout basics

- Layout components
  - Column
    - place items vertically on the screen
  - Row
    - place items horizontally on the screen
  - Box
    - put elements on top of another
    - supports the configuration of specific alignments of its elements



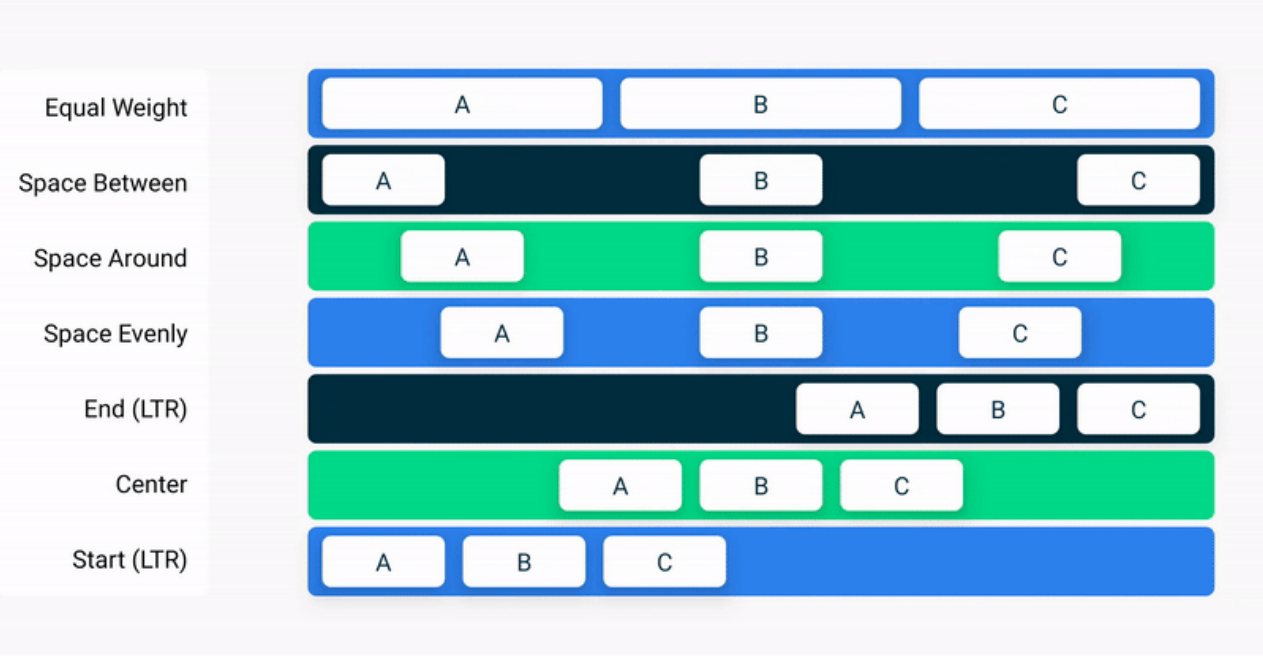
Column, Row and Box

```
@Composable
inline fun Column(
    modifier: Modifier = Modifier,
    verticalArrangement: Arrangement.Vertical = Arrangement.Top,
    horizontalAlignment: Alignment.Horizontal = Alignment.Start,
    content: @Composable ColumnScope.() -> Unit
): Unit
```



Row

```
@Composable
inline fun Row(
    modifier: Modifier = Modifier,
    horizontalArrangement: Arrangement.Horizontal = Arrangement.Start,
    verticalAlignment: Alignment.Vertical = Alignment.Top,
    content: @Composable RowScope.() -> Unit
): Unit
```



Box and Spacer

```
@Composable
fun Box(modifier: Modifier): Unit
```

```
@Composable
inline fun Box(
    modifier: Modifier = Modifier,
    contentAlignment: Alignment = Alignment.TopStart,
    propagateMinConstraints: Boolean = false,
    content: @Composable BoxScope.() -> Unit
): Unit
```

```
@Composable
@NonRestartableComposable
fun Spacer(modifier: Modifier): Unit
```

```
Row {
    Box(Modifier.size(100.dp).background(Color.Red))
    Spacer(Modifier.width(20.dp))
    Box(Modifier.size(100.dp).background(Color.Magenta))
    Spacer(Modifier.weight(1f))
    Box(Modifier.size(100.dp).background(Color.Black))
}
```

Other layouts

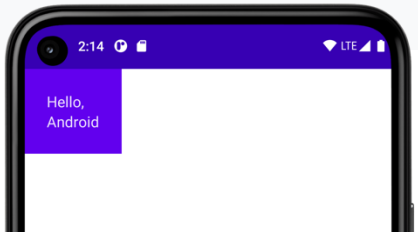
- Pager
- Flow Layouts
- BoxWithConstraints
- Constraint Layout
- Custom Layouts

Modifiers

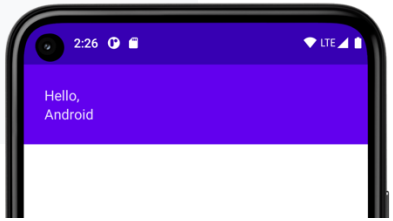
- Modifiers allow to decorate or augment a composable

- Modifiers can…
  - Change the composable's size, layout, behavior, and appearance
  - Add information, like accessibility labels
  - Process user input
  - Add high-level interactions, like making an element clickable, scrollable, draggable, or zoomable

```
@Composable
private fun Greeting(name: String) {
    Column(modifier = Modifier.padding(24.dp)) {
        Text(text = "Hello,")
        Text(text = name)
    }
}
```




```
@Composable
private fun Greeting(name: String) {
    Column(
        modifier = Modifier
            .padding(24.dp)
            .fillMaxWidth()
    ) {
        Text(text = "Hello,")
        Text(text = name)
    }
}
```



Modifiers

- The order of *modifiers* matters

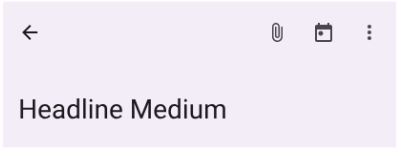
```
@Composable
fun ArtistCard(/*...*/) {
    val padding = 16.dp
    Column(
        Modifier
            .clickable(onClick = onClick)
            .padding(padding)
            .fillMaxWidth()
    ) {
        // rest of the implementation
    }
}
```

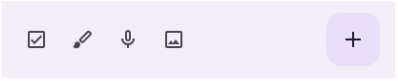
```
@Composable
fun ArtistCard(/*...*/) {
    val padding = 16.dp
    Column(
        Modifier
            .padding(padding)
            .clickable(onClick = onClick)
            .fillMaxWidth()
    ) {
        // rest of the implementation
    }
}
```

https://developer.android.com/jetpack/compose/modifiers-list

Other components

- `Scaffold`
  - Allows to define a common mobile application with top bar, bottom bar and floating action button
- App Bars
  - `TopAppBar`, `BottomAppBar`








```
@Composable
fun Scaffold(
    modifier: Modifier = Modifier,
    scaffoldState: ScaffoldState = rememberScaffoldState(),
    topBar: @Composable () -> Unit = {},
    bottomBar: @Composable () -> Unit = {},
    snackbarHost: @Composable (SnackbarHostState) -> Unit = { SnackbarHost(it) },
    floatingActionButton: @Composable () -> Unit = {},
    floatingActionButtonPosition: FabPosition = FabPosition.End,
    isFloatingActionButtonDocked: Boolean = false,
    drawerContent: (@Composable ColumnScope.() -> Unit)? = null,
    drawerGesturesEnabled: Boolean = true,
    drawerShape: Shape = MaterialTheme.shapes.large,
    drawerElevation: Dp = DrawerDefaults.Elevation,
    drawerBackgroundColor: Color = MaterialTheme.colors.surface,
    drawerContentColor: Color = contentColorFor(drawerBackgroundColor),
    drawerScrimColor: Color = DrawerDefaults.scrimColor,
    backgroundColor: Color = MaterialTheme.colors.background,
    contentColor: Color = contentColorFor(backgroundColor),
    content: @Composable (PaddingValues) -> Unit
): Unit
```

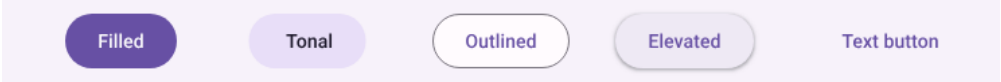
```
@Composable
fun TopAppBar(
    title: @Composable () -> Unit,
    windowInsets: WindowInsets,
    modifier: Modifier = Modifier,
    navigationIcon: (@Composable () -> Unit)? = null,
    actions: @Composable RowScope.() -> Unit = {},
    backgroundColor: Color = MaterialTheme.colors.primarySurface,
    contentColor: Color = contentColorFor(backgroundColor),
    elevation: Dp = AppBarDefaults.TopAppBarElevation
): Unit
```

Material 3

```
@Composable
public fun Scaffold(
    modifier: Modifier = Modifier,
    topBar: @Composable () -> Unit = {},
    bottomBar: @Composable () -> Unit = {},
    snackbarHost: @Composable () -> Unit = {},
    floatingActionButton: @Composable () -> Unit = {},
    floatingActionButtonPosition: FabPosition = FabPosition.End,
    containerColor: Color = MaterialTheme.colorScheme.background,
    contentColor: Color = contentColorFor(containerColor),
    contentWindowInsets: WindowInsets = ScaffoldDefaults.contentWindowInsets,
    content: @Composable (PaddingValues) -> Unit
): Unit
```

Other components

- Buttons




```
@Composable
fun Button(
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    enabled: Boolean = true,
    shape: Shape = ButtonDefaults.shape,
    colors: ButtonColors = ButtonDefaults.buttonColors(),
    elevation: ButtonElevation? = ButtonDefaults.buttonElevation(),
    border: BorderStroke? = null,
    contentPadding: PaddingValues = ButtonDefaults.ContentPadding,
    interactionSource: MutableInteractionSource = remember { MutableInteractionSource() },
    content: @Composable RowScope.() -> Unit
): Unit
```

```
@Composable
fun OutlinedButton(
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    enabled: Boolean = true,
    shape: Shape = ButtonDefaults.outlinedShape,
    colors: ButtonColors = ButtonDefaults.outlinedButtonColors(),
    elevation: ButtonElevation? = null,
    border: BorderStroke? = ButtonDefaults.outlinedButtonBorder,
    contentPadding: PaddingValues = ButtonDefaults.ContentPadding,
    interactionSource: MutableInteractionSource = remember { MutableInteractionSource() },
    content: @Composable RowScope.() -> Unit
): Unit
```

Text

```
@Composable
fun BigText() {
    Text("Hello World", fontSize = 30.sp)
}
```

```
@Composable
fun BoldText() {
    Text("Hello World", fontWeight = FontWeight.Bold)
}
```

```
@Composable
fun TextShadow() {
    val offset = Offset(5.0f, 10.0f)
    Text(
        text = "Hello world!",
        style = TextStyle(
            fontSize = 24.sp,
            shadow = Shadow(
                color = Color.Blue, offset = offset, blurRadius = 3f
            )
        )
    )
}
```

Hello world!

```
@Composable
fun PasswordTextField() {
    var password by rememberSaveable { mutableStateOf("") }

    TextField(
        value = password,
        onValueChange = { password = it },
        label = { Text("Enter password") },
        visualTransformation = PasswordVisualTransformation(),
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password)
    )
}
```

Enter password
•••••

```
@Composable
fun SimpleFilledTextFieldSample() {
    var text by remember { mutableStateOf("Hello") }

    TextField(
        value = text,
        onValueChange = { text = it },
        label = { Text("Label") }
    )
}
```

Label
Hello

```
@Composable
fun SimpleOutlinedTextFieldSample() {
    var text by remember { mutableStateOf("") }

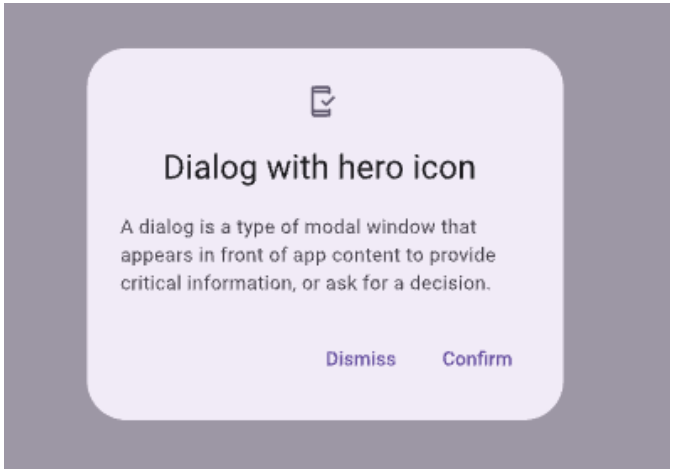
    OutlinedTextField(
        value = text,
        onValueChange = { text = it },
        label = { Text("Label") }
    )
}
```

Label
Hello Compose

https://developer.android.com/jetpack/compose/text/fonts

Other components

- Dialog / AlertDialog



- Progress indicators

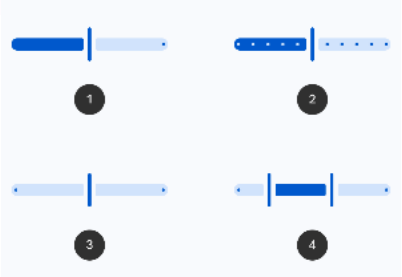


- Sliders



- Switches





List and grids

- Lazy Lists

```
LazyColumn {
    // Add a single item
    item {
        Text(text = "First item")
    }

    // Add 5 items
    items(5) { index ->
        Text(text = "Item: $index")
    }

    // Add another single item
    item {
        Text(text = "Last item")
    }
}
```

```
LazyRow(
    horizontalArrangement = Arrangement.spacedBy(4.dp),
) {
    // ...
}
```

```
@Composable
fun MessageList(messages: List<Message>) {
    Column {
        messages.forEach { message ->
            MessageRow(message)
        }
    }
}
```

- Lazy Grids, Lazy staggered grid

```
LazyVerticalGrid(
    columns = GridCells.Adaptive(minSize = 128.dp)
) {
    items(photos) { photo ->
        PhotoItem(photo)
    }
}
```

```
LazyVerticalStaggeredGrid(
    columns = StaggeredGridCells.Adaptive(200.dp),
    verticalItemSpacing = 4.dp,
    horizontalArrangement = Arrangement.spacedBy(4.dp),
    content = {
        items(randomSizedPhotos) { photo ->
            AsyncImage(
                model = photo,
                contentScale = ContentScale.Crop,
                contentDescription = null,
                modifier = Modifier.fillMaxWidth().wrapContentHeight()
            )
        }
    },
    modifier = Modifier.fillMaxSize()
)
```

Exercise

- *Create an application that allows users to draw lines of different colors using touch gestures (finger)*

- Create a new *Jetpack Compose* "Empty Activity" project
- Remove the unnecessary `Greeting` composable
- Move the `Scaffold` composable to a separate file, making it a global composable function

Exercise

- Add a `TopBar` to the `Scaffold`, including:
  - Title (e.g., "`PaintAMov`")
  - Set the title container color using the RGB values (255, 255, 224)
  - Action buttons for selecting the color of the lines to be drawn
    - Available colors: red, green, blue, and black (other colors can also be chosen)
    - Width: 40dp
    - Aspect ratio: 1:1
    - Padding: 4dp



- Add a `remember` variable to store the currently selected color

- Test the buttons by changing the color of the title

Exercise

- Add the following classes to the `model` package of the project (create the package if it doesn't already exist)

```
data class Point(val x: Float, val y: Float)

data class Line(val begin: Point, val end: Point, val color: ULong)

class Drawing {
    private val lines = mutableListOf<Line>()

    val _linesFlow = MutableStateFlow<List<Line>>(lines.toList())
    val linesFlow : StateFlow<List<Line>> = _linesFlow

    fun addLine(line: Line) {
        lines.add(line)
        _linesFlow.value = lines.toList()
    }
}
```

Exercise

- Add a View Model to the project
  - Include the *View Model for Compose* library in the module dependencies:

```
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")
```

  - Add a `DrawingViewModel` class (extending `ViewModel`) to the project
    - Create an instance of the model (`Drawing`)
    - Observe and transform the model data to the format required by the project's composables

```
data class CanvasLine(val begin: Offset, val end: Offset, val color: Color)

class DrawingViewModel : ViewModel() {
    private val drawing = Drawing()

    private val _linesFlow = MutableStateFlow<List<CanvasLine>>(emptyList())
    val linesFlow: MutableStateFlow<List<CanvasLine>> = _linesFlow

    init {
        viewModelScope.launch {
            drawing.linesFlow.collect { lines ->
                _linesFlow.emit(lines.map { line ->
                    CanvasLine( Offset(line.begin.x, line.begin.y),
                                Offset(line.end.x, line.end.y),     Color(line.color) )
                })
            }
        }
    }
}
```

Exercise

- Add a method, `addLine`, to the `DrawingViewModel` to add a new line to the drawing

```
fun addLine(
      start: Offset,
      end: Offset,
      color : Color
) {
    drawing.addLine(
        Line(
            Point(start.x, start.y),
            Point(end.x, end.y),
            color.value
        )
    )
}
```

Exercise

- Create and pass by parameter the *View Model* to the `MainScreen` Composable

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PaintAMovTheme {
                MainScreen(viewModel())
            }
        }
    }
}

fun MainScreen(
    viewModel: DrawingViewModel,
    modifier: Modifier = Modifier
) {
   ...
}
```

Exercise

- Add a `Canvas` composable to the body of the `Scaffold` in `MainScreen` to detect gestures for creating and drawing new lines

```
Scaffold (
    …
) { innerPadding ->
        Canvas(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
        ) { }
}
```

Exercise

- Add a `pointerInput` modifier to the `Canvas` composable to detect gestures
  - In this case, drag gestures will be used to draw the lines

```
<canvas modifier>
   .pointerInput(Unit) {
      detectDragGestures { change, dragAmount ->
         change.consume()
         viewModel.addLine(
            change.position-dragAmount,
            change.position,
            currentColor
         )
      }
   },
```

Exercise

- Draw the lines defined in the application model data
  - Add a variable to make the composable aware of the lines in the model
    - This variable allows the flow to be observed as a state

```
val linesFlow by viewModel.linesFlow.collectAsState()
```

    - Draw each line in the `Canvas`

```
Canvas ( ... ) {
    linesFlow.forEach { line ->
        drawLine(
            start = line.begin,
            end = line.end,
            color = line.color,
            strokeWidth = 10f
        )
    }
}
```

Exercise

- Add a "Clear" button to the application to delete all drawn lines
  - The button should include an appropriate icon followed by the text "Clear"

